

SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU
FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I
INFORMACIJSKIH TEHNOLOGIJA OSIJEK

Stručni prijediplomski studij Računarstvo

GENERATOR PUTOVANJA

Završni rad

Petar Ruškan

Osijek, 2024.

SADRŽAJ

1. UVOD.....	1
1.1. Opis problema.....	1
1.2. Cilj završnog rada.....	1
1.3. Važnost teme	1
1.4. Pregled rezultata iz citirane literature	1
1.5. Struktura završnog rada.....	2
1.5.1. Pregled područja teme	2
1.5.2. Dizajn i implementacija sustava.....	2
1.5.3. Testiranje i evaluacija.....	2
1.5.4. Zaključak	2
1.6. Zadatak rada.....	2
2. PREGLED PODRUČJA TEME	3
2.1. Uvod	3
2.2. Automatizacija planiranja putovanja	3
2.2.1. Prikupljanje podataka	3
2.2.2. Analiza podataka	3
2.2.3. Prikaz podataka	4
2.2.4. Implementacija algoritma preporuke	4
2.3. Postojeće aplikacije i sustavi.....	4
2.3.1. Google Trips	4
2.3.2. TripAdvisor	5
2.3.3. Kayak	6
2.3.4. Analiza aplikacija.....	6
2.4. Tehnologije u automatizaciji planiranja putovanja	7
2.4.1. Angulara u izradi korisničkih sučelja	7
2.4.2. Tailwind CSS.....	7
2.4.3. NestJs za razvoj back-end sustava.....	8
2.4.4. MongoDB kao baza podataka	9
2.4.5. Visual Studio Code.....	10
3. DIZAJN I IMPLEMENTACIJA SUSTAVA	11
3.1. Struktura baze podataka	11
3.2. Početna stranica	13

3.3. Prijava korisnika.....	14
3.4. Registracija korisnika	17
3.5. Generator stranica	19
3.5.1. Mapa.....	19
3.5.2. Generiranje putovanja	22
3.5.3. Spremljena putovanja	29
3.6. Profil.....	30
3.7. Javna mjesta.....	32
3.8. Kontakt stranica	34
4. ZAKLJUČAK.....	36
LITERATURA	38
SAŽETAK	39
ABSTRACT	40
ŽIVOTOPIS	41
PRILOZI.....	42

1. UVOD

1.1. Opis problema

U današnjem ubrzanom načinu života planiranje putovanja može biti izazovno i dugotrajno. Većina ljudi traži učinkovit način organiziranja putovanja koji odgovara njihovim osobnim preferencijama i budžetu. Tradicionalne metode planiranja putovanja, kao što je savjetovanje s putničkim agentom ili pretraživanje brojnih *web* stranica, često su složene, nefleksibilne i dugotrajne. S obzirom na rastuću potražnju za personaliziranim i jednostavnim rješenjima, postoji potreba za automatiziranim alatom koji može generirati putovanja na temelju specifičnih korisničkih kriterija.

1.2. Cilj završnog rada

Cilj ovog rada je razviti *web* aplikaciju pod nazivom Generator putovanja koja korisnicima omogućuje kreiranje putovanja prema njihovim željama. Aplikacija će korisnicima dati mogućnost odabira vrste putovanja (zabava ili razgledavanje) te početne, završne lokacije te mjesta između početne i krajnje lokacije. *Web* aplikacija koristi Angular kao *front-end* razvojni okvir, NestJs kao *back-end* i MongoDB kao bazu podataka.

1.3. Važnost teme

Tema je važna jer adresira potrebu za digitalizacijom i automatizacijom procesa planiranja putovanja, čineći ga pristupačnijim i prilagođenijim korisnicima. Razvoj ovakve aplikacije može značajno smanjiti vrijeme i trud potrebne za planiranje putovanja, pružajući korisnicima personalizirana rješenja na jednostavan i intuitivan način. Također, upotreba modernih tehnologija kao što su Angular, NestJs i MongoDB omogućava stvaranje skalabilnog, fleksibilnog i sigurnog sustava.

1.4. Pregled rezultata iz citirane literature

Dosadašnja istraživanja i razvoj u području automatizacije planiranja putovanja pokazala su da primjena informacijske tehnologije u ovom području ima brojne prednosti. Istraživanja

pokazuju da korisnici preferiraju personalizirane i prilagodljive alate koji integriraju različite aspekte planiranja putovanja, uključujući smještaj i aktivnosti. Korištenje naprednih algoritama za preporuke i analitike podataka također se pokazalo korisnim u poboljšanju točnosti i relevantnosti predloženih itinerera [1]. Kombinacija *front-end* i *back-end* tehnologija kao što su Angular i NestJs s MongoDB bazom podataka pruža solidnu osnovu za razvoj takvih aplikacija, omogućujući brzu obradu podataka i fleksibilnost u dizajnu korisničkog sučelja.

1.5. Struktura završnog rada

Ovaj završni rad podijeljen je u nekoliko poglavlja, kako slijedi:

1.5.1. Pregled područja teme

U ovom poglavlju bit će prikazan pregled relevantne literature i trenutnog stanja u području automatizacije planiranja putovanja, uključujući pregled sličnih aplikacija i tehnologija koje se koriste.

1.5.2. Dizajn i implementacija sustava

Ovo poglavlje obuhvaća detaljan opis arhitekture sustava, uključujući odabir tehnologija i njihovu integraciju.

1.5.3. Testiranje i evaluacija

Ovo poglavlje će opisati metode testiranja aplikacije, rezultate testiranja te evaluaciju performansi i korisničkog iskustva.

1.5.4. Zaključak

Završno poglavlje sažima rezultate rada, daje preporuke za budući razvoj i ističe ključne zaključke.

1.6. Zadatak rada

Web aplikaciju automatski stvara planove putovanja za korisnike. Aplikacija pomaže korisnicima u organizaciji svojih putovanja pružajući sugestije za destinacije, aktivnosti, raspored, i druge detalje. Generator putovanja koristi korisničke preferencije, datume putovanja, i druge faktore kako bi personalizirao predložene planove, čime olakšava planiranje.

2. PREGLED PODRUČJA TEME

2.1. Uvod

Automatizacija planiranja putovanja je postala sve popularnija tema u informacijskoj tehnologiji, s ciljem poboljšanja korisničkog iskustva i učinkovitost u organizaciji putovanja. U ovom pregledu su obuhvaćeni trenutni trendovi, tehnologije i aplikacije koje se koriste u ovoj domeni.

2.2. Automatizacija planiranja putovanja

Kompleksan proces koji koristi napredne tehnologije i algoritme kako bi olakšala korisnicima planiranje njihovih putovanja. Ovaj proces obuhvaća nekoliko ključnih koraka: prikupljanje podataka, njihovu analizu i na kraju prikaz tih podataka na korisnikovom ekranu.

2.2.1. Prikupljanje podataka

Prvi korak u automatizaciji planiranja putovanja je prikupljanje podataka. Podaci su prikupljeni pomoću API-ja (engl. *Application Programming Interface*) za turističke informacije. Jedan od najčešće korištenih alata za prikupljanje ovih podataka je Google *Places* API. Ovaj API omogućava pristup informacijama o različitim turističkim atrakcijama, restoranima, hotelima i drugim važnim mjestima interesa koje bi mogle biti relevantne za planiranje putovanja. Podaci prikupljeni putem Google *Places* API-ja mogu uključivati detaljne opise lokacija, radno vrijeme, popularnost, cijene i korisničke recenzije.

2.2.2. Analiza podataka

Nakon prikupljanja podataka, slijedi faza njihove analize. U ovoj fazi koriste se različiti algoritmi za obradu i analizu podataka kako bi se identificirale najbolje opcije za putovanje. Na primjer, algoritmi mogu analizirati ocijenu mjesta kako bi se procijenila kvaliteta određenih

atrakcija ili usluga, te kombinirati ove podatke s informacijama o udaljenosti i dostupnosti kako bi se generiralo putovanje.

2.2.3. Prikaz podataka

Konačno, rezultati analize prezentirani su korisnicima na intuitivan i lako razumljiv način. Jedan od alata koji se često koristi za vizualizaciju ovih podataka je Maplibre-gl, otvoreni alat za mapiranje koji može interaktivno prikazati geografske informacije. Uz Maplibre-gl korisnici mogu vidjeti preporučene rute, turističke atrakcije, restorane i druge važne točke interesa na interaktivnoj karti. Ovaj vizualni prikaz može uključivati različite razine informacija, kao što su dobivene lokacije za posjetiti, ruta od najdalje sve do najbliže te detaljan opis svake lokacije.

2.2.4. Implementacija algoritma preporuke

Osim vizualizacije, ključnu ulogu u automatizaciji planiranja putovanja ima i implementacija algoritama preporuka. Ovi algoritmi uzimaju u obzir korisničke postavke kao što su odabir mjesta koje korisnik želi posjetiti, interesi, radijus pretraživanja mjesta za generiranje personaliziranih preporuka. Na primjer, ako korisnik preferira kulturne atrakcije, algoritmi će analizirati dostupne podatke i preporučiti muzeje, galerije ili povijesne lokacije koje se uklapaju u te kriterije.

2.3. Postojeće aplikacije i sustavi

Trenutno na tržištu postoji mnoštvo aplikacija koje nude usluge planiranja putovanja, među kojima se ističu Google *Trips*, TripAdvisor i Kayak. Svaka od ovih aplikacija koristi različite pristupe kako bi pružila informacije korisnicima, što se kreće od osnovnih funkcionalnosti poput pretraživanja i rezervacije smještaja ili prijevoza, pa sve do implementacije naprednih algoritama za personalizaciju putovanja.

2.3.1. Google Trips

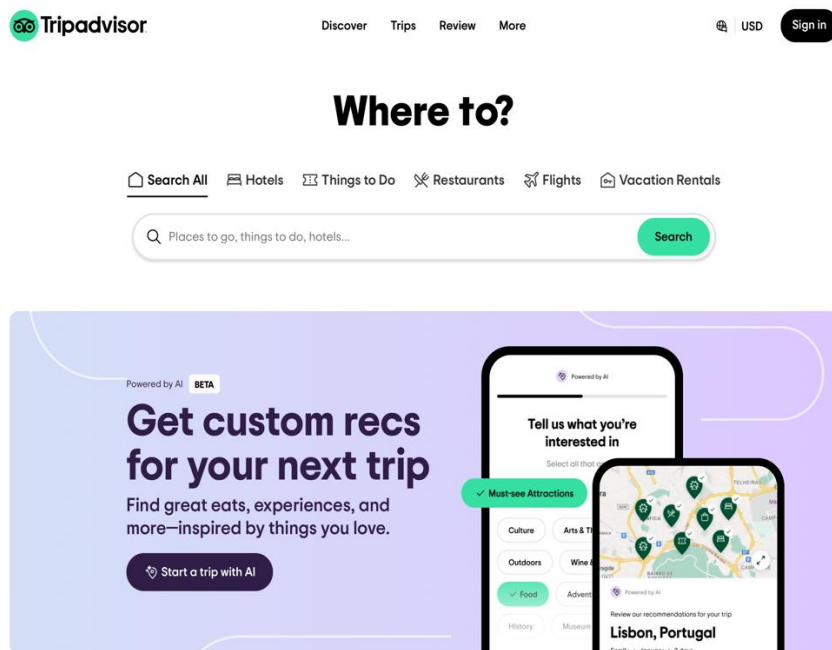
Google *Trips* omogućava korisnicima da automatski organiziraju svoje rezervacije i pronalaze lokalne atrakcije. Aplikacija prikuplja podatke iz Gmail računa korisnika kako bi automatski generirala itinerere, uključujući informacije o letovima, smještaju, najmu automobila i rezervacijama za aktivnosti. Također, korisnici mogu pregledavati preporučene planove putovanja i atrakcije u blizini njihove destinacije, što omogućava jednostavno i intuitivno planiranje putovanja [2]. Slika 2.3.1. Prikazuje Google *Trips*.



Slika 2.1. Google Trips

2.3.2. TripAdvisor

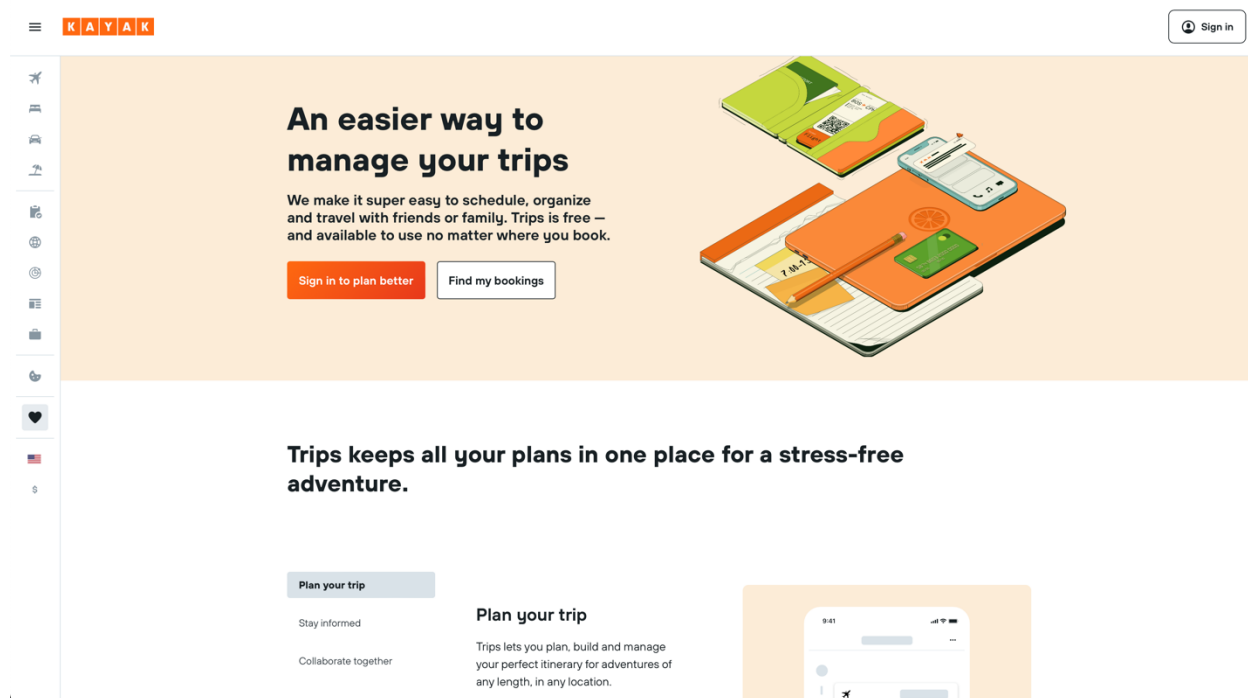
TripAdvisor koristi veliki broj korisničkih recenzija i ocjena kako bi preporučio najbolja mjesta za posjetiti. Ova aplikacija omogućava korisnicima pretraživanje i rezervaciju smještaja, restorana, letova i aktivnosti. S velikim brojem recenzija, TripAdvisor pruža detaljne informacije o kvaliteti usluga i atrakcija, čime korisnicima pomaže u donošenju informiranih odluka. Personalizirane preporuke temelje se na povijesti pretraživanja i ocjenama korisnika, što doprinosi boljem korisničkom iskustvu [3]. Slika 2.3.2. Prikazuje TripAdvisor.



Slika 2.2. TripAdvisor

2.3.3. Kayak

Kayak se fokusira na pretraživanje i usporedbu cijena letova, hotela i *rent-a-car* usluga, pružajući korisnicima širok spektar opcija za planiranje njihovih putovanja. Aplikacija koristi napredne algoritme za pretraživanje i filtriranje rezultata, omogućujući korisnicima da pronađu najpovoljnije ponude. Također, Kayak nudi alate za praćenje cijena i obavijesti o promjenama cijena, čime pomaže korisnicima da uštede novac prilikom rezervacije [4]. Slika 2.3.3. Prikazuje Kayak.



Slika 2.3. Kayak

2.3.4. Analiza aplikacija

Analiza ovih aplikacija može pružiti vrijedan uvid u najbolje prakse u industriji, kao i identificirati moguće nedostatke koje naš projekt, Generator putovanja, može poboljšati. Na primjer, dok neke aplikacije nude visok stupanj personalizacije, mogu biti ograničene u pogledu integracije različitih vrsta putovanja ili fleksibilnosti u postavljanju budžeta. Kroz detaljno proučavanje postojećih rješenja možemo identificirati ključne funkcionalnosti koje korisnici cijene, kao i područja gdje postoji prostor za inovaciju i poboljšanje, što će omogućiti našoj aplikaciji da pruži superiorno korisničko iskustvo.

2.4. Tehnologije u automatizaciji planiranja putovanja

Razvoj aplikacija za planiranje putovanja obuhvaća širok spektar tehnologija, uključujući *front-end* razvojni okvir (*Framework*) kao što je Angular, *back-end* razvojni okvir poput NestJs, te baze podataka kao što su MongoDB. Svaka od ovih tehnologija nudi specifične prednosti i izazove, a njihov izbor često ovisi o zahtjevima projekta i preferencijama razvojnih timova.

2.4.1. Angulara u izradi korisničkih sučelja

Angular je jedan od najpopularnijih *front-end* razvojni okvir za izradu dinamičnih *web* aplikacija, razvijen od strane Googlea. Njegova modularna arhitektura omogućava razdvajanje kôda u različite module, olakšavajući organizaciju i održavanje kôda, što je posebno korisno za složene aplikacije poput „Generatora putovanja“. Angular nudi snažnu podršku za reaktivno programiranje putem RxJS biblioteke, što omogućava učinkovito rukovanje asinkronim operacijama kao što su dohvaćanje podataka s poslužitelja i korisnički unosi [5].

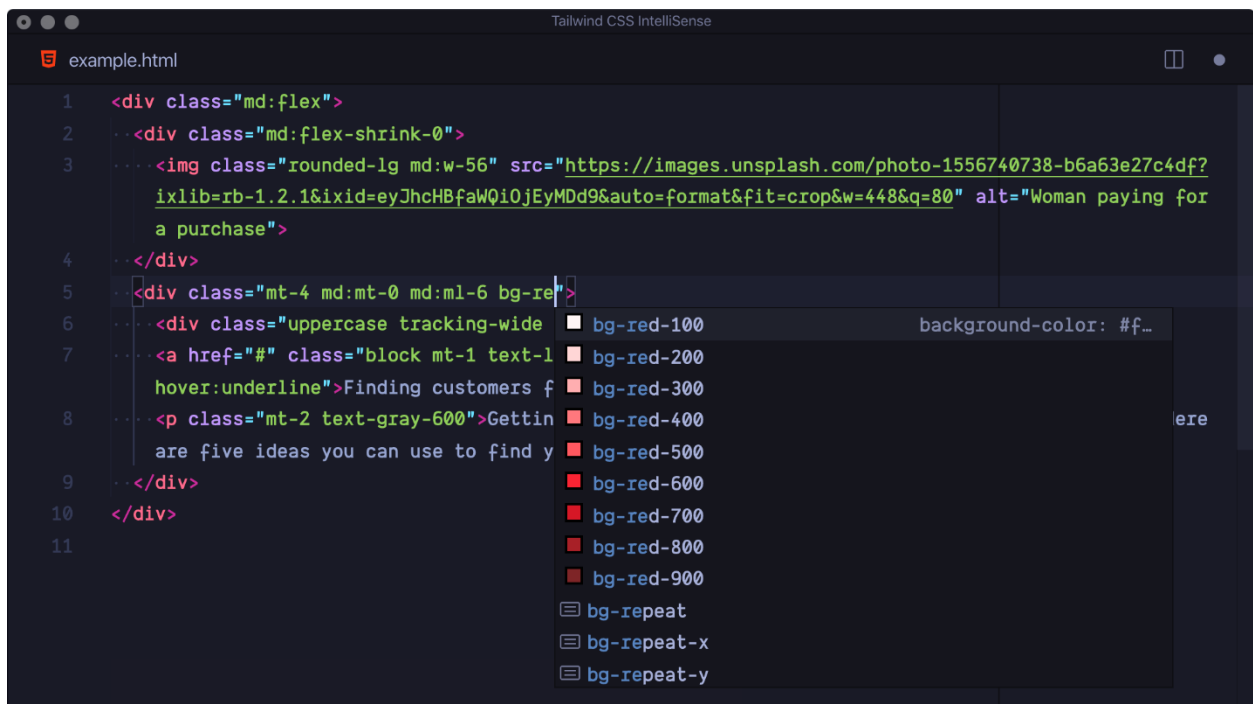
Dvosmjerno vezivanje podataka automatski sinkronizira model i prikaz, smanjujući potrebu za ručnim ažuriranjem DOM-a (engl. *Document Object Model*) i smanjujući mogućnost grešaka. Jednostavna integracija s različitim tehnologijama i alatima, poput API servisa i autentifikacijskih sustava, omogućava sigurno i efikasno slanje i dohvaćanje podataka između klijenta i poslužitelja. Angular također omogućava izradu visoko interaktivnih i responzivnih korisničkih sučelja, pružajući izvrsno korisničko iskustvo kroz ugrađene direktive i alate za manipulaciju DOM-om [6]. Uz to, Angular CLI (engl. *Command Line Interface*) alat skraćuje vrijeme potrebno za razvoj, dok alat za testiranje, poput Lint, osiguravaju kvalitetu kôda i smanjuju broja pogrešaka. Sve ove značajke čine Angular idealnim izborom za izradu moderne, skalabilne i responzivne *web* aplikacije, kao što je projekt „Generator putovanja“.

2.4.2. Tailwind CSS

Tailwind CSS (engl. *Cascading Style Sheets*) je *utility-first* CSS okvir koji omogućava izradu prilagodljivih korisničkih sučelja na učinkovit i brz način. Umjesto pisanja prilagođenih CSS pravila za svaku komponentu, Tailwind koristi unaprijed definirane klase koje se izravno primjenjuju unutar HTML (engl. *Hyper Text Markup Language*) elemenata. Ovaj pristup omogućava programerima da kombiniranjem jednostavnih klasa, poput ``text-center``, ``bg-blue``

500`, ili `p-4`, oblikuju sučelje bez potrebe za ručnim pisanjem dodatnih CSS stilova. Prednost ovog pristupa je povećana produktivnost te veća fleksibilnost u dizajnu sučelja.

Jedna od ključnih prednosti Tailwind CSS-a je njegova modularnost i skalabilnost. Dolazi s ugrađenom podrškom za responzivnim dizajnom, omogućujući jednostavnu prilagodbu korisničkog sučelja različitim veličinama ekrana putem svojih *breakpoint* klasa, kao što su `sm:`, `md:`, `lg:` i `xl:`. Ova fleksibilnost u kombinaciji s mogućnošću brze iteracije čini Tailwind CSS jednim od najpoželjnijih alata u modernom razvoju *front-end* aplikacija. Slika 2.6. prikazuje primjer korištenja Tailwindcss-a [7].

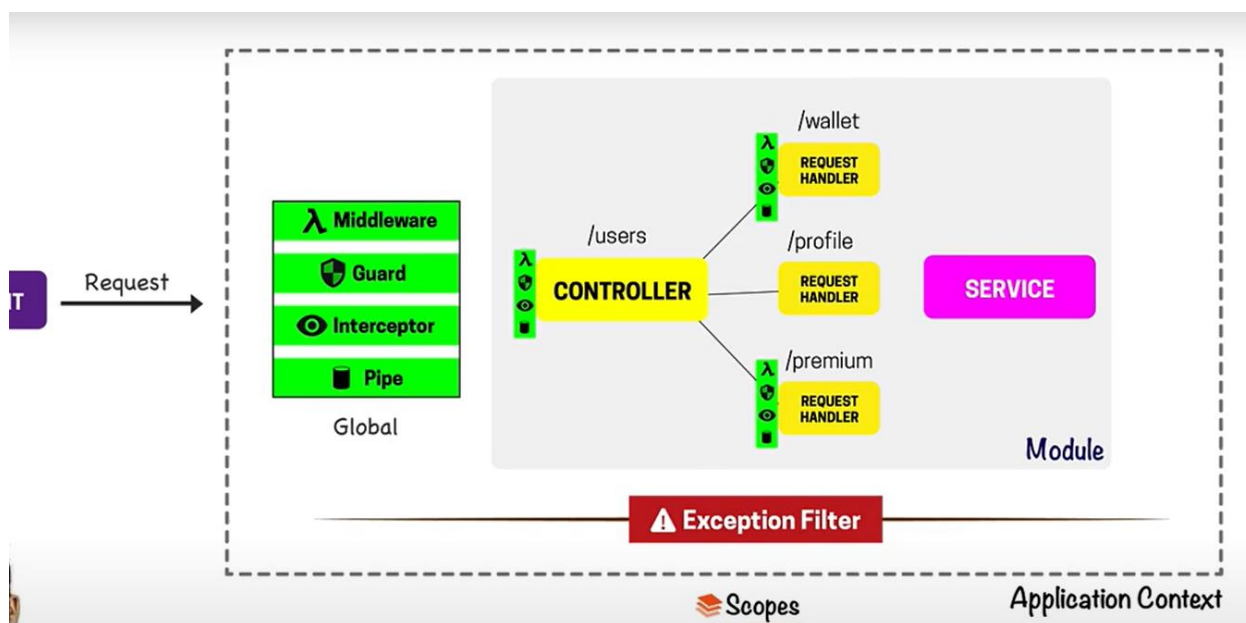


Slika 2.4. Primjer korištenja Tailwindcss-a

2.4.3. NestJs za razvoj back-end sustava

Korištenje NestJS-a s Angularom moćna je kombinacija za razvoj modernih web aplikacija s visokim performansama i skalabilnošću. NestJS je pozadinski okvir temeljen na TypeScriptu inspiriran dizajnom i načelima Angulara, koji pomaže u razvoju kompletnih aplikacija koristeći isti jezik i alate kao Angular. Jedan od ključnih razloga za korištenje NestJS-a s Angularom je njihova kompatibilnost u smislu tehnoloških skupova. Oba okvira koriste TypeScript, omogućujući zajedničko korištenje tipova podataka, sučelja i drugih TypeScript značajki. Time se smanjuje mogućnost pogrešaka pri komunikaciji između *front-end*-a i *back-end*-a jer se na oba kraja koristi isti jezik. Drugi važan faktor je upotreba istih ili sličnih dizajn principa i

arhitektonskih obrazaca. NestJS je baziran na konceptima kao što su *dependency injection*, moduli, *providera* i *middleware*-a, što je slično pristupu koji se koristi u Angularu za organizaciju kôda. Ovaj paralelizam olakšava razumijevanje i održavanje kôda u obje strane aplikacije [8]. Slika 2.7. prikazuje NestJs arhitekturu.



Slika 2.5. NestJs arhitektura

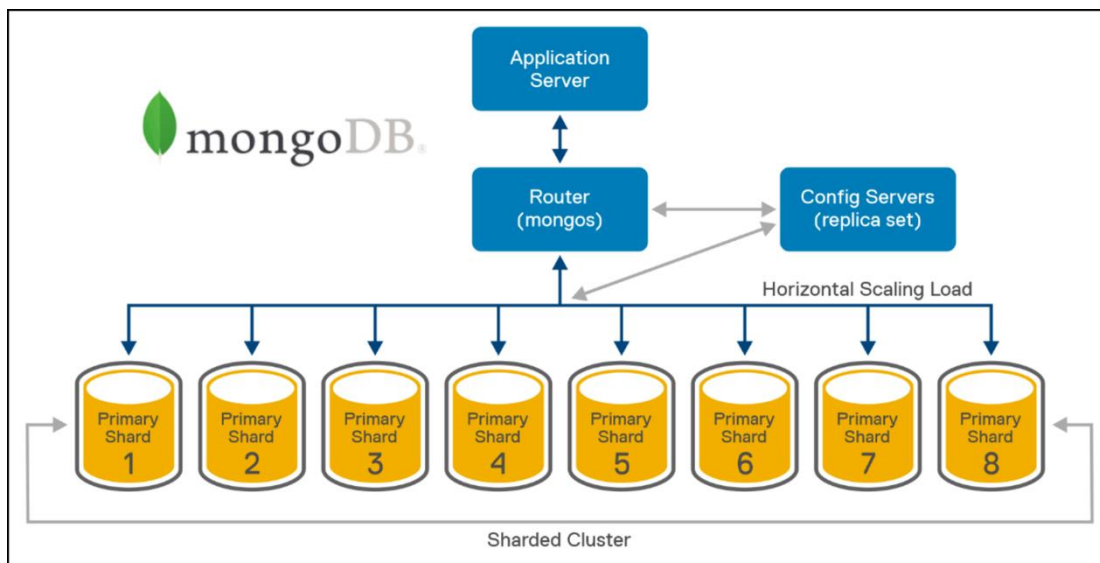
Izvor: [NestJS Overview and Architecture + Installation](#)

2.4.4. MongoDB kao baza podataka

MongoDB je *NoSQL* baza podataka koja se ističe svojom fleksibilnošću i skalabilnošću, što je čini idealnim izborom za aplikacije koje zahtijevaju brzo pretraživanje i obradu velikih količina podataka. Kao dokument-orijentirana baza podataka, MongoDB pohranjuje podatke u formatu sličnom JSON-u (engl. *JavaScript Object Notation*), omogućavajući programerima jednostavno modeliranje složenih struktura podataka [9].

Ova karakteristika posebno dolazi do izražaja u aplikacijama gdje se podaci ne uklapaju lako u tradicionalne relacijske sheme. Pružajući horizontalnu skalabilnost, MongoDB podržava distribuciju podataka preko više servera, čime se povećava dostupnost i otpornost sustava. Njegova mogućnost jednostavne integracije s *back-end* sustavima razvijenim u NestJs-u omogućava učinkovitu komunikaciju između aplikacijskih slojeva, što doprinosi bržem razvoju i pouzdanijem radu aplikacije. Pored toga, MongoDB podržava napredne funkcionalnosti kao što su indeksiranje, agregacija podataka i replikacija, što dodatno povećava performanse i pouzdanost sustava.

Korištenjem MongoDB-a u projektu „Generator putovanja“, osiguravamo da baza podataka može učinkovito rukovati velikim brojem korisničkih zahtjeva i kompleksnih upita, pružajući optimalno korisničko iskustvo i pouzdanost podataka. Slika 2.8. Prikazuje MongoDB arhitekturu.



Slika 2.6. MongoDB arhitektura

Izvor: [MongoDB architecture](#)

2.4.5. Visual Studio Code

Visual Studio Code (VS Code) besplatan je uređivač otvorenog izvornog kôda koji je razvio Microsoft i koji je brzo postao popularan među programerima zbog svoje jednostavnosti i fleksibilnosti. VS Code podržava više programskih jezika kao što su JavaScript, Python, C# itd., što ga čini idealnim alatom za razne projekte. Podržava različite dodatke u različitim programskim jezicima i prilagodbu sučelja različitim temama.

VS Code dolazi s mnogim integriranim alatima kao što su *debugger*, terminal i podrška za *linting* i automatsko formatiranje kôda. Njegova integracija s platformama kao što su GitHub i GitLab omogućuje lakše upravljanje kôdom u timskom okruženju. Osim toga, sa značajkom IntelliSense korisnici dobivaju naprednu pomoć s automatskim prijedlozima i ispravcima dok pišu kôd.

3. DIZAJN I IMPLEMENTACIJA SUSTAVA

Za izradu aplikacije korištene su suvremene tehnologije web developmenta. *Back-end* dio razvijen je korištenjem NestJS razvojni okvira, unutar kojeg su stvorene MongoDB sheme za entitete korisnik i trip. Implementirani su kontroleri i servisi koji pokrivaju ključne funkcionalnosti aplikacije, poput autentifikacije, generiranja putovanja, slanja kontakt informacija te upravljanja podacima korisnika. Autentifikacija je implementirana korištenjem JWT-a (engl. JSON Web Token), što omogućuje siguran prijenos podataka i autorizaciju unutar sustava. JWT se sastoji od tri dijela: zaglavlja koje sadrži tip tokena i algoritam enkripcije, tijela s informacijama korisnika (npr. korisnički ID, vrijeme isteka sesije) te potpisa koji osigurava integritet podataka.

Front-end dio aplikacije razvijen je korištenjem Angular razvojni okvira. Korisničko sučelje sadrži komponente za prijavu, registraciju, pregled i generiranje putovanja, uključujući integraciju s kartama. U front-end dijelu implementirane su validacije unosa podataka pri prijavi i registraciji kako bi se osigurala ispravnost i sigurnost korisničkih podataka.

3.1. Struktura baze podataka

Struktura baze podataka aplikacije temelji se na MongoDB kolekcijama koje koriste definirane sheme unutar NestJS razvojni okvira. Dvije ključne sheme su *User* i *ITrip*, koje definiraju korisničke podatke i putovanja.

Shema *ITrip* sadrži attribute: *tripId*, kao jedinstveni identifikator putovanja i automatski se generira prilikom stvaranja novog putovanja, *tripName*, obavezan atribut koji predstavlja naziv putovanja. *Places* je lista lokacija koje korisnik planira posjetiti, a svaki element liste predstavlja pojedinu lokaciju pohranjenu kao objekt. Atribut *comment* omogućava vlasniku dodavanje komentara o putovanju, dok atribut *public* označava je li putovanje javno ili privatno. *Likes* bilježi broj “lajkova” koje je putovanje primilo, dok *likedBy* pohranjuje listu ID-eva korisnika koji su “lajkali” putovanje. Konačno, *timeSaved* označava trenutni datum spremanja putovanja. Na Slici 3.1. prikazana je *ITrip* shema unutar NestJs komponente.

```

@Schema()
export class ITrip {
  @Prop({ type: Types.ObjectId, default: () => new Types.ObjectId() })
  tripId: Types.ObjectId;

  @Prop({ type: String, required: true })
  tripName: string;

  @Prop({ type: [mongoose.Schema.Types.Mixed] })
  places: Place[];

  @Prop({ type: String })
  comment: string;

  @Prop({ type: Boolean, default: false })
  public: boolean;

  @Prop({ type: Number })
  likes: number;

  @Prop({ type: [Types.ObjectId], ref: 'User', default: [] })
  likedBy: Types.ObjectId[];

  @Prop({ type: Date, default: Date.now })
  timeSaved: Date;
}

```

Slika 3.1. *ITrip* shema

Shema *User* sadrži attribute: *role*, koji označava ulogu korisnika u sustavu i zadano je postavljen na *user*, dok *admin* ima role *admin*, *username* i *email*, koji moraju biti jedinstveni i obavezni prilikom registracije. Atribut *password* pohranjuje hashiranu lozinku korisnika. *FirstName* i *lastName* su opcionalni atributi koji pohranjuju ime i prezime korisnika. Atribut *tokens* prati broj tokena koje korisnik ima, a njegova početna vrijednost iznosi 5 tokena. *Trips* predstavlja listu putovanja povezanih s korisnikom, pri čemu svako putovanje koristi *TripSchema* kao referencu. Na Slici 3.2. prikazana je *User* shema unutar NestJs komponente.

```

@Schema()
export class User extends Document {
  @Prop({ required: true, default: 'user' })
  role: string;

  @Prop({ required: true, unique: true })
  username: string;

  @Prop({ required: true, unique: true })
  email: string;

  @Prop({ required: true })
  password: string;

  @Prop({ required: false, default: '' })
  firstName: string;

  @Prop({ required: false, default: '' })
  lastName: string;

  @Prop({ required: true, default: 5 })
  tokens: number;

  @Prop({ type: [TripSchema], default: [] })
  trips: ITrip[];

  async comparePassword(enteredPassword: string): Promise<boolean> {
    return bcrypt.compare(enteredPassword, this.password);
  }
}

```

Slika 3.2. User schema

Na slici 3.3. prikazano je spajanje na bazu podataka sa *back-end*-a iz *app.module.ts* datoteke. *MONGO_URI* se dodaje unutar *.env* datoteke. Datoteka *.env* je konfiguracijska datoteka u kojoj se pohranjuju osjetljive informacije poput API ključeva, lozinki i drugih varijabli okruženja koje aplikacija koristi tijekom izvođenja. Na slici je prikazan primjer MongoDB *connection string*-a.

```

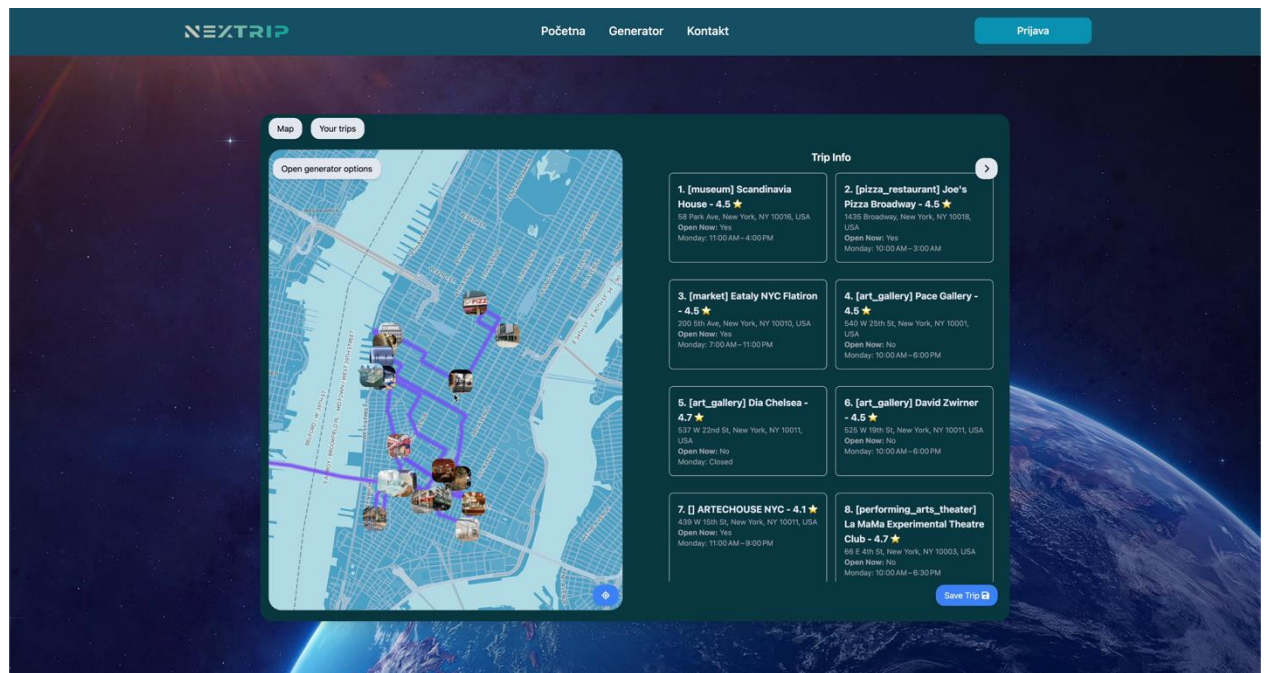
MONGO_URI=mongodb+srv://USERNAME:PASSWORD@nextrip.vra6w97.mongodb.net/NEXTTRIP?
retryWrites=true&w=majority&appName=nextrip

```

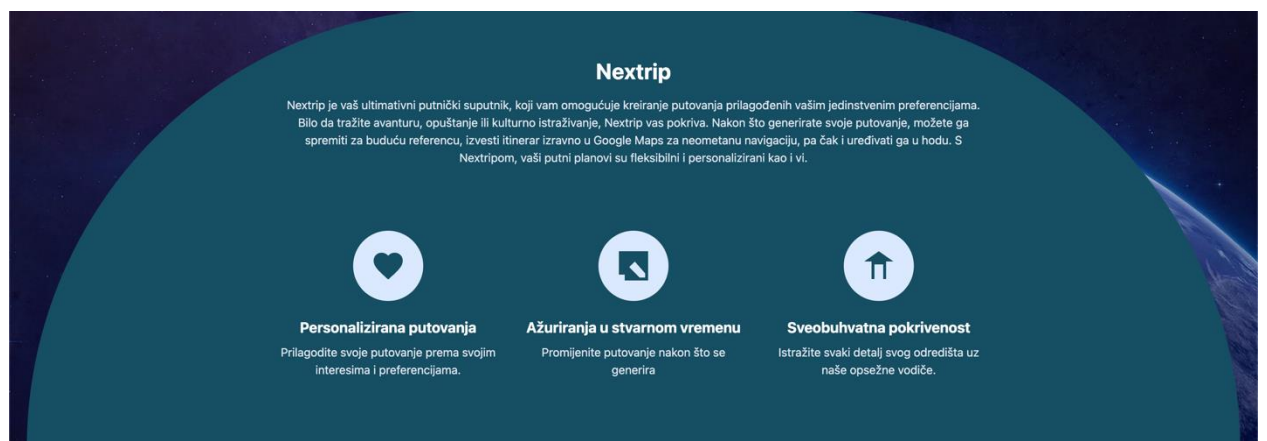
Slika 3.3. MongoDB connection string

3.2. Početna stranica

Na slici 3.4. i 3.5. je prikazana početna stranica koje sadrži navigaciju sa nazivom i logom *web* aplikacije, u sredini su tri dugmeta: Početna, generator i kontakt, dok na desnoj strani stoje dugme za prijavu, te ispod navigacije video i opis aplikacije.



Slika 3.4. Navigacija i video početne stranice

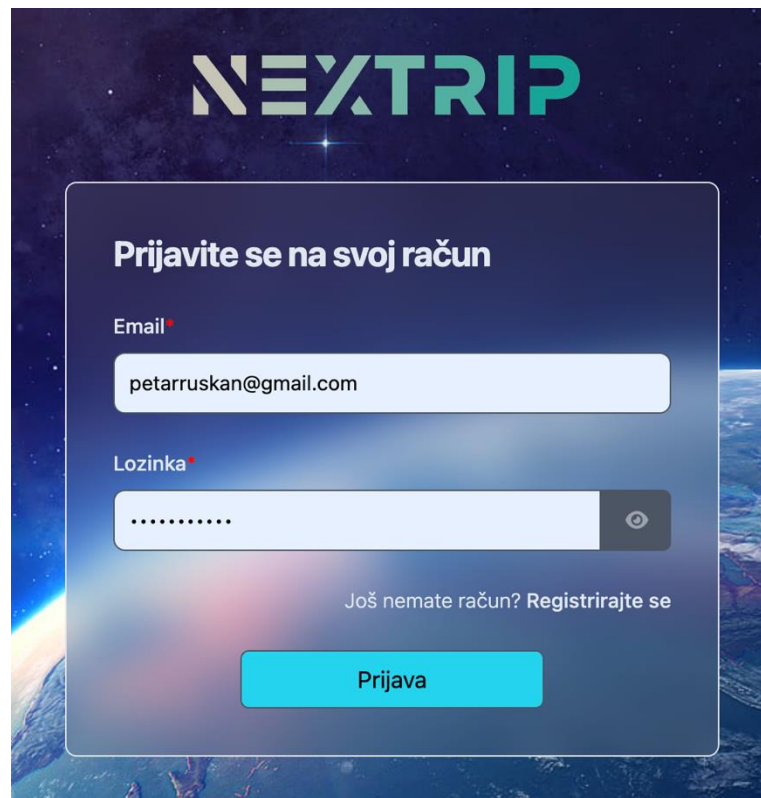


Slika 3.5. Dodatne značajke aplikacije

Video prikazuje kako koristiti aplikaciju za korisnike koji po prvi put koriste. Ispod videa su značajke koje dodatno objašnjavaju aplikaciju.

3.3. Prijava korisnika

Slika 3.6. prikazuje dio sa prijavom koje se sastoji od jednostavnog angular obrazca koji sadrži unos za *e-mail* i lozinku.



Slika 3.6. Izgled obrasca za prijavu

Nakon što se unese *e-mail* i lozinka pritiskom na dugme prijava, podatci se preko angular servisa za autentifikaciju šalju na *back-end*.

Slika 3.7. prikazuje login funkciju u *auth.service.ts* koja šalje podatke na *back-end*. Koristi *HTTP* (engl. *Hypertext Transfer Protocol*) POST zahtjev za slanje *e-mail* adrese i lozinke na */auth/login endpoint*. Kada server odgovori s JWT tokenom, metoda poziva funkciju *validateToken* za provjeru valjanosti tokena. Ako token prođe validaciju, pohranjuje se u *localStorage* pod nazivom *jwt_token* a metoda vraća *true*, što označava da je prijava uspješna. U slučaju neuspješne validacije tokena, metoda vraća *false*.

Ako tijekom prijave dođe do greške, metoda poziva `logout`, briše token iz lokalne pohrane. U tom slučaju, pomoću `catchError` funkcije osigurava se da aplikacija ne prekine s radom, te se vraća `Observable` s vrijednošću *false*, što znači da prijava nije uspjela. Ovaj pristup omogućava robustno upravljanje prijavom korisnika.

```

public login(email: string, password: string): Observable<boolean> {
  return this.http
    .post<{
      token: string;
    }>(`${this.BACKEND_API}/auth/login`, { email, password })
    .pipe(
      map((response) => {
        const tokenPayload = this.validateToken(response.token);
        this.localStorage.setItem('jwt_token', response.token);
        return !!tokenPayload;
      }),
      catchError((error) => {
        this.logOut();
        console.error('Login error:', error);
        return of(false);
      })
    );
}

```

You, 4 months ago • Added login component

Slika 3.7. Login funkcija unutar *front-end auth.service.ts*

Na slici 3.8. je prikazana asinkronu metodu za prijavu korisnika koja vraća *Promise* koji može sadržavati JWT token ili *null*. Metoda prima korisnički *e-mail* i lozinku kao argumente i prvo traži korisnika u bazi podataka koristeći *e-mail* adresu. Ako korisnik postoji i lozinka unesena pri prijavi odgovara *hashiranoj* lozinki spremljenoj u bazi (provjereno pozivom metode *comparePassword*), metoda generira i vraća JWT token koji identificira korisnika.

Ako korisnik ne postoji ili lozinke ne odgovaraju, metoda vraća *null*, što označava da prijava nije uspjela. Ovaj pristup omogućava sigurno provjeravanje vjerodajnica korisnika i generiranje tokena za uspješne prijave.

```

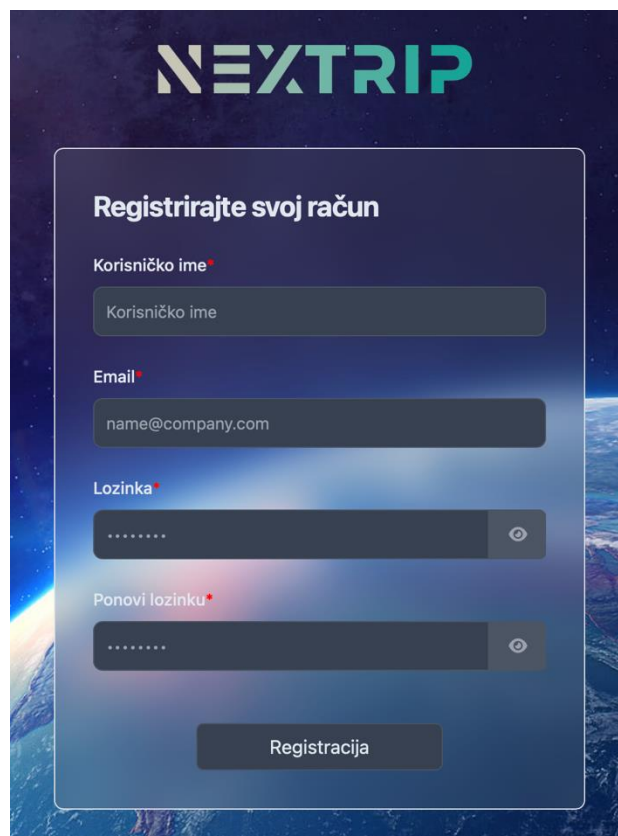
async login(email: string, password: string): Promise<string | null> {
  const user = await this.userModel.findOne({ email });
  if (user && (await user.comparePassword(password))) {
    return this.createToken(user._id as Types.ObjectId);
  }
  return null;
}

```

Slika 3.8. Login funkcija unutar *back-end auth.service.ts*

3.4. Registracija korisnika

Na stranici 3.9. za registraciju korisnika, *front-end* dizajn koristi Angular komponente za prikaz jednostavnog, ali funkcionalnog obrasca za registraciju. Forma se sastoji od 4 polja: Korisničko ime, *e-mail*, lozinka i Ponovi lozinku. Svako polje ima implementirane validacije koje provjeravaju ispravnost unesenih podataka pomoću *regex*-a prije slanja obrasca. U slučaju neispravnog unosa, korisnik se odmah obavještava o grešci kroz vizualne indikatore, poput poruka o pogrešci koje se dinamički prikazuju ispod svakog polja.



Slika 3.9. Izgled obrasca za registraciju

Kada korisnik ispravno ispuni sva polja i pošalje obrazac, *front-end* šalje podatke na *back-end* server za autentifikaciju. Slika 3.10. prikazuje metodu *register* koja se koristi za registraciju korisnika. Metoda prima tri argumenta: *username*, *email*, i *password*, te vraća *Observable* koji emitira *boolean* vrijednost. Unutar metode, poziva se HTTP POST zahtjev na *endpoint* za registraciju na serveru, koristeći URL koji se sastoji od osnovnog URL-a *backend* API-ja i putanje */auth/register*. Tijekom slanja zahtjeva, šalje se objekt koji sadrži unesene podatke korisnika. Nakon što se primi odgovor, metoda koristi *map* operator za provjeru valjanosti JWT tokena koji se nalazi u odgovoru. Token se pohranjuje u lokalnu pohranu preglednika, a

metoda vraća *true* ako je token valjan, ili *false* u suprotnom. Ako dođe do pogreške tijekom registracije, metoda će vratiti *false*.

```
public register(  
  username: string,  
  email: string,  
  password: string,  
) : Observable<boolean> {  
  return this.http  
    .post<{  
      token: string;  
   }>(`${this.BACKEND_API}/auth/register`, { username, email, password })  
    .pipe(  
      map((response) => {  
        const tokenPayload = this.validateToken(response.token);  
        this.localStorage.setItem('jwt_token', response.token);  
        return !!tokenPayload;  
      }),  
      catchError((error) => {  
        console.error('Registration error:', error);  
        return of(false);  
      })  
    );  
}
```

Slika 3.10. Register funkcija unutar *front-end auth.service.ts*

Slika 3.11. Prikazuje asinkronu *register* funkciju koja prima korisničko ime, *e-mail* i lozinku kao ulazne podatke i vraća token kao rezultat. Ako korisnik ne postoji, stvara se novi korisnik s primljenim podacima i provodi se validacija tih podataka. Nakon što se podaci validiraju, novi korisnik se sprema u bazu podataka. Na kraju se generira i vraća token za novog korisnika.

```

async register(
  username: string,
  email: string,
  password: string,
): Promise<string> {
  try {
    const existingUser = await this.userModel.findOne({
      $or: [{ email }, { username }],
    });
    if (existingUser) {
      throw new ConflictException('User already exists');
    }

    const user = new this.userModel({ username, email, password });
    await user.validate();
    await user.save();

    return this.createToken(user._id as Types.ObjectId);
  } catch (error) {
    Logger.error('Failed to register user', error.stack);
    throw new InternalServerErrorException('Failed to register user');
  }
}

```

Slika 3.11. Register funkcija unutar *back-end auth.service.ts*

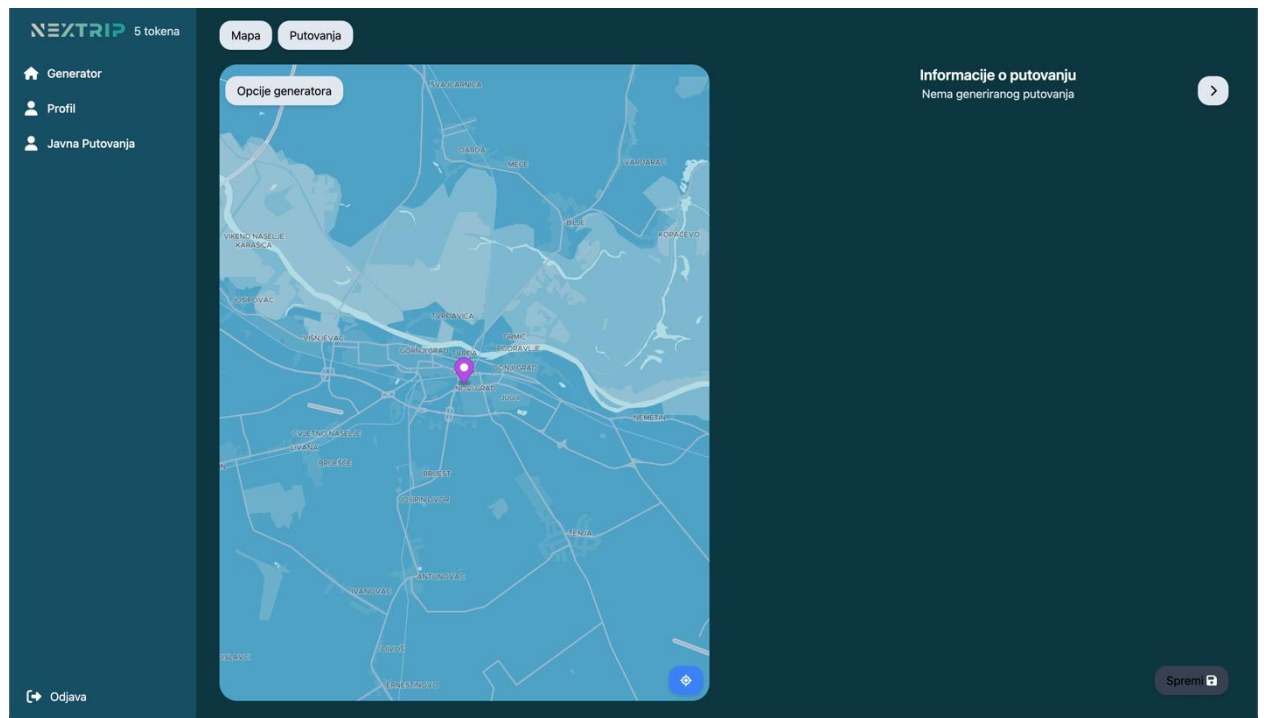
3.5. Generator stranica

Nakon uspješne prijave korisniku se otvara generator stranica. Generator stranica je *child* komponenta od *sidebar*-a. *Sidebar* je *parent* komponenta također *profile*-u i *public trip*-u. *Sidebar* sadrži dugme za odjavu korisnika.

Generator stranica se sastoji od *sidebar*-a na lijevoj strani, i generator komponente na desnoj. Generator komponenta se sastoji od tri dijela. *Parent* komponenta sa dva dugmeta za prikaz mape, kao na slici te spremljenih putovanja. Dvije *child* komponente *Map* i *Your Trips*.

3.5.1. Mapa

Na slici 3.12. vidimo *child* komponentu mapa koja je prikazana uz pomoć MapLibreGL biblioteke. Funkcija koja stvara mapu je prikazana na slici 3.13. Mapa je glavna značajka aplikacije jer korisnik postiže interakciju sa njom. Mapa sadrži dugme Opcije generatora te dugme kako bi se mapa pozicionirala na njihovu lokaciju. Na desnoj strani od mape se nalaze Informacije o putovanju koje se mogu sakriti kako bi se mapa povećala.



Slika 3.12. Prikaz *child* komponente Mapa

```

async initializeMap() {
  try {
    await this.setLocation();

    if (this.initialState) {
      this.map = new Map({
        container: this.mapContainer.nativeElement,
        style: `${this.MAP_STYLE_JSON}?apiKey=${this.MAP_STYLE_API}`,
        zoom: 11,
        minZoom: 2,
        center: this.initialState,
        fadeDuration: 0,
        attributionControl: false,
      });

      const layers = layersFactory();
      layers.push({
        id: 'maplibre-gl-directions-waypoint-label',
        type: 'symbol',
        source: 'maplibre-gl-directions',
        layout: {
          'text-field': [
            'case',
            ['==', ['get', 'category'], 'ORIGIN'],
            'A',
            ['==', ['get', 'category'], 'DESTINATION'],
            'B',
            '',
          ],
        },
        paint: {
          'text-color': '#ffffff',
          'text-opacity': 0.7,
        },
        filter: [
          'all',
          ['==', ['geometry-type'], 'Point'],
          ['==', ['get', 'type'], 'WAYPOINT'],
          ['in', ['get', 'category'], ['literal', ['ORIGIN', 'DESTINATION']]],
        ],
      });

      this.setUserMarker();

      this.map.on('load', () => {
        this.mapContainer.nativeElement.classList.remove('loader');
        this.directions = new CustomMapLibreGldirections(this.map, {
          requestOptions: {
            alternatives: 'false',
            overview: 'full',
            steps: 'true',
          },
          layers,
        });

        this.directions.interactive = true;
        this.map.addControl(new LoadingIndicatorControl(this.directions));

        this.directions.on('addwaypoint', (e) => {
          if (e.data && e.data.index !== undefined) {
            if (e.data.index >= 5) {
              alert('Max 5 locations');
              this.directions.removeWaypoint(e.data.index);
            }

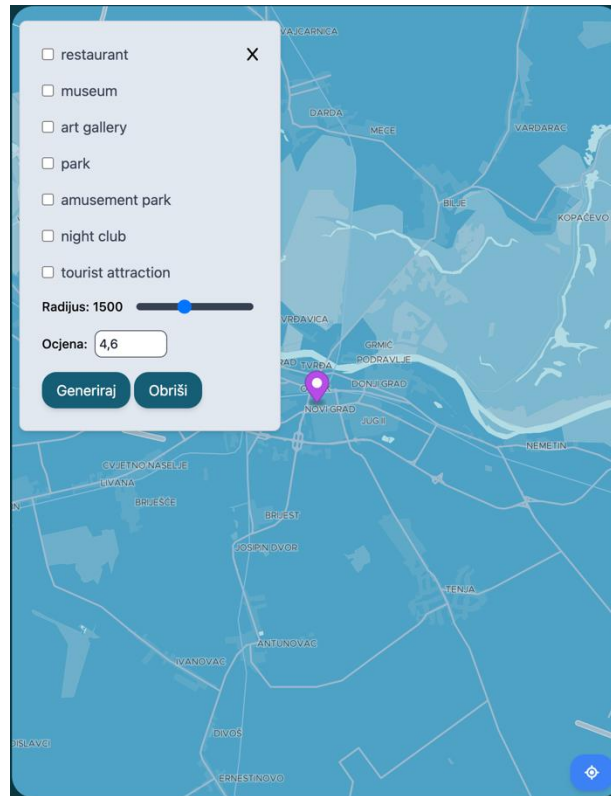
            if (this.editMode) {
              alert('Cannot add locations in edit mode');
              this.directions.removeWaypoint(e.data.index);
            }
          }
        });
      });
    } catch (error) {
      console.error(error);
    }
  }
}

```

Slika 3.13. Funkcija initializeMap za prikaz mape

3.5.2. Generiranje putovanja

Na slici 3.14. je prikazan obrazac koji se otvara nakon pritiska na dugme „Opcije generatora“ te u kojemu korisnik odabire različite mogućnosti koje želi na svome putovanju. U isto vrijeme su odabrana mjesta koje bi korisnik želio posjetiti.



Slika 3.14. Obrazac opcije generatora

Nakon slanja odabranih opcija i lokacija, podatci se šalju preko `generator.service.ts` na *back-end*. Slika 3.15. prikazuje funkciju `generateRoutePrivate` koja služi za generiranje rute koristeći privatne podatke. Metoda prima tri parametra: `waypointsFeatures`, koji je niz objekata tipa `Feature<Point, GeoJsonProperties>`, `generatorParams`, koji sadrži parametre za generiranje rute, te `userId`, koji predstavlja jedinstveni identifikator korisnika. Metoda koristi HTTP POST zahtjev za slanje podataka na API endpoint definiran u ``BACKEND_API`` s dodatkom `/generator`. Podaci koji se šalju uključuju `waypointsFeatures`, `generatorParams`, i `userId`. Rezultat ove HTTP POST operacije je *observable* koji se očekuje da vrati niz objekata tipa `Place`.

```
private generateRoutePrivate(
  waypointsFeatures: Feature<Point, GeoJsonProperties>[],
  generatorParams: generatorParams,
  userId: string,
): Observable<Place[]> {
  return this.http.post<Place[]>(`${this.BACKEND_API}/generator`, {
    waypointsFeatures: waypointsFeatures,
    generatorParams: generatorParams,
    userId: userId,
  });
}
```

Slika 3.15. *generateRoutePrivate* funkcija unutar *front-end generator.service.ts*

Slika 3.16. prikazuje objekte *generatorParams* i *Place*.

```
export interface generatorParams {
  typeOfTrip: {
    restaurant: boolean | null | undefined;
    museum: boolean | null | undefined;
    art_gallery: boolean | null | undefined;
    park: boolean | null | undefined;
    amusement_park: boolean | null | undefined;
    night_club: boolean | null | undefined;
    tourist_attraction: boolean | null | undefined;
  };
  radius: number | null;
  rating: number | null;
}

export interface Place {
  displayName: {
    text: string;
  };
  formattedAddress?: string;
  location?: {
    latitude: number;
    longitude: number;
  };
  primaryType: string;
  rating: number;
  websiteUri: string;
  googleMapsUri: string;
  currentOpeningHours?: {
    openNow: boolean;
    periods: {
      open: {
        day: number;
        time: string;
      };
      close?: {
        day: number;
        time: string;
      };
    };
  }[];
  weekdayDescriptions?: string[];
};
photos?: {
  name: string;
  widthPx: number;
  heightPx: number;
  authorAttributions?: {
    displayName: string;
    uri: string;
  }[];
}[];
}
```

Slika 3.16. Objekti *generatorParams* i *Place*

Slika 3.17. prikazuje funkciju *generateTrip* u *back-end*-u koja prima podatke sa *front-end*-a te asinkrono generira putovanje na temelju zadanih točaka (*waypoints*), parametara generatora i korisničkog ID-a. Prvo dohvaća korisnika iz baze podataka pomoću korisničkog ID-a. Ako korisnik ne postoji, baca se greška „*User not found*“. Također, provjerava se ima li korisnik dovoljno tokena za generiranje putovanja; ako nema, baca se greška „*Insufficient tokens*“.

```
async generateTrip(
  waypoints: Feature<Point, GeoJsonProperties>[],
  generatorParams: TripDto,
  userId: string,
): Promise<Place[]> {
  try {
    const user = await this.userModel.findById(userId);
    if (!user) {
      throw new Error('User not found');
    }

    if (user.tokens <= 0) {
      throw new Error('Insufficient tokens');
    }

    const trueKeys = this.getTrueKeys(generatorParams);

    const responses = await Promise.all(
      waypoints.map(async (waypoint) => {
        const [longitude, latitude] = this.getCoordinates(waypoint);
        const radius = generatorParams.radius;

        return radius < 1500
          ? await this.makeSingleApiCall(
              latitude,
              longitude,
              radius,
              trueKeys,
            )
          : await this.makeMultipleApiCalls(
              latitude,
              longitude,
              radius,
              trueKeys,
            );
      }),
    );

    const filteredPlaces = this.filterAndMapPlaces(
      generatorParams,
      responses.flat(),
    );

    const combinedPlaces = this.combinePlaces(
      filteredPlaces,
      waypoints.length,
    );

    const lastWaypoint = waypoints[waypoints.length - 1];
    const sortedPlaces = this.sortPlacesByDistanceFromWaypoint(
      combinedPlaces,
      lastWaypoint,
    );

    user.tokens -= 1;
    await user.save();

    return sortedPlaces;
  } catch (error) {
    Logger.error(
      'Error fetching nearby places',
      error.message,
      'GeneratorService',
    );
    if (error.response) {
      Logger.error(
        'API Response Error: ${error.response.data}',
        null,
        'GeneratorService',
      );
    }
    throw new HttpException(
      'Error fetching nearby places',
      HttpStatus.INTERNAL_SERVER_ERROR,
    );
  }
}
```

Slika 3.17. *generateTrip* funkcija unutar *back-end* *generator.service.ts*

Nakon toga, funkcija dohvaća sve aktivirane parametre iz objekta *generatorParams* i koristi ih za pozive API-a. Za svaku točku na putovanju, izdvajaju se koordinate (*longitude* i *latitude*), te se na temelju radijusa odlučuje hoće li se napraviti jedan ili više API poziva. Slika 3.18. prikazuje primjer pozivanja *makeSingleApiCall*. Ako je radijus manji od 1500, obavlja se jedan poziv, inače se obavlja više poziva kako bi se pokrilo veće područje.

```
private async makeSingleApiCall(
  latitude: number,
  longitude: number,
  radius: number,
  trueKeys: string[],
): Promise<Place[]> {
  const requestBody = {
    includedTypes: trueKeys,
    maxResultCount: 20,
    locationRestriction: {
      circle: {
        center: { latitude, longitude },
        radius,
      },
    },
    rankPreference: 'POPULARITY',
  };

  const response = await axios.post(
    GOOGLE_PLACES_SEARCH_NEARBY_URL,
    requestBody,
    {
      headers: this.getRequestHeaders(),
    },
  );

  return response.data.places || [];
}
```

Slika 3.18. *makeSingleApiCall* funkcija unutar *back-end generator.service.ts*

Dobiveni odgovori se filtriraju i mapiraju u liste objekata *Place*, te se kombiniraju u jedinstvenu listu, nakon čega se sortiraju prema udaljenosti od zadnje točke na putovanju. Na kraju, korisnički tokeni se smanjuju za jedan i korisnički podaci se pohranjuju u bazu. Ako dođe do greške, ona se bilježi i baca se HTTP iznimka s porukom "*Error fetching nearby places*" te statusom 500 (*Internal Server Error*). Slika 3.19. prikazuje funkciju koja sortira mjesta od najdaljeg sve do najbližeg određenoj lokaciji.

```

private sortPlacesByDistanceFromWaypoint(
  places: Place[],
  waypoint: Feature<Point, GeoJsonProperties>,
): Place[] {
  const [waypointLng, waypointLat] = this.getCoordinates(waypoint);

  return places.sort((a, b) => {
    const distanceA = this.calculateDistance(
      waypointLat,
      waypointLng,
      a.location.latitude,
      a.location.longitude,
    );
    const distanceB = this.calculateDistance(
      waypointLat,
      waypointLng,
      b.location.latitude,
      b.location.longitude,
    );
    return distanceB - distanceA;
  });
}

private calculateDistance(
  lat1: number,
  lng1: number,
  lat2: number,
  lng2: number,
): number {
  const R = 6371;
  const dLat = this.deg2rad(lat2 - lat1);
  const dLng = this.deg2rad(lng2 - lng1);
  const a =
    Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(this.deg2rad(lat1)) *
    Math.cos(this.deg2rad(lat2)) *
    Math.sin(dLng / 2) *
    Math.sin(dLng / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return R * c;
}

private deg2rad(deg: number): number {
  return deg * (Math.PI / 180);
}

```

Slika 3.18. Funkcije za sortiranje mjesta unutar *back-end generator.service.ts*

Slika 3.19. prikazuje postupak prikazivanja mjesta na mapu. Ovaj kôd prikazuje funkciju *handleTripGenerationSuccess* koja se aktivira nakon uspješne generacije putovanja, pri čemu uzima kao parametar odgovor u obliku niza objekata *Place*. Unutar funkcije, odgovor se dodjeljuje varijabli *places*. Zatim se poziva funkcija za promjenu parametara modalnog prozora.

Sljedeći korak je čišćenje trenutnih ruta u *directions* objektu i postavljanje njegove interaktivnosti na *false* kako korisnik ne bi mogao slučajno dodati nešto. Iz odgovora se filtriraju lokacije koje sadrže koordinate, a zatim se njihove geografske širine i dužine mapiraju u niz *waypoints*. Nakon toga, poziva se funkcija za postavljanje markera na lokacijama, dok se karta pomiče prema prvoj

lokaciji iz niza i postavlja na odgovarajuće povećanje. Na kraju, *directions* objekt postavlja dobivene lokacije kao rute.



```
private handleTripGenerationSuccess(response: Place[]): void {
  this.places = response;
  this.cdr.markForCheck();

  this.toggleModalParams();

  this.directions.clear();
  this.directions.interactive = false;

  const waypoints: [number, number][] = response
    .filter(
      (
        locationObj,
      ): locationObj is Place & {
        location: { longitude: number; latitude: number };
      } => !!locationObj.location,
    )
    .map((locationObj) => [
      locationObj.location.longitude,
      locationObj.location.latitude,
    ]);

  this.setPlaceMarker();
  this.map.flyTo({
    center: waypoints[0],
    zoom: 11,
  });
  this.directions.setWaypoints(waypoints);
}
```

Slika 3.19. Funkcija *handleTripGenerationSuccess* unutar *front-end map.component.ts*

Funkcija *handleTripGenerationSuccess* poziva *setPlaceMarker* koja stvara markere na mapi za svaku dobivenu lokaciju. Svaki marker kada se otvori prikazuje detaljan opis te slike mjesta koje korisnik može pregledati. Slika 3.20. prikazuje funkciju *setPlaceMarker*.

```

setPlaceMarker(): void {
  this.places?.forEach((place, index) => {
    if (place.location) {
      const el = document.createElement('div');
      el.className = 'marker';

      const photoReference = place.photos?.[0]?.name.split('/').pop();
      const imageUrl = photoReference
        ? `https://maps.googleapis.com/maps/api/place/photo?
maxwidth=60&photo_reference=${photoReference}&key=${environment.GOOGLE_PLACES_API}`
        : `https://picsum.photos/seed/${index + 1}/60/60`;

      el.style.backgroundImage = `url(${imageUrl})`;
      el.style.width = '45px';
      el.style.height = '45px';
      el.style.backgroundColor = 'cover';
      el.style.borderRadius = '30%';
      el.style.display = 'inline-block';
      el.style.overflow = 'hidden';

      const popupContent = `
<div class="p-4 max-w-xs">
  <button
    id="remove-point-${index}"
    class="absolute left-0 top-0 px-2 py-1 bg-slate-200 text-black shadow-md focus:outline-none z-40
hover:bg-cyan-800 hover:text-white transition-transform duration-300 transform hover:scale-95"
  >
    Remove Point
  </button>
  <strong class="text-lg">${index + 1}. ${
    place.displayName.text
  }</strong><br>
  <strong>Rating:</strong> ${place.rating} ★<br>
  <strong>Address:</strong> ${place.formattedAddress || 'N/A'}<br>
  <strong>Website:</strong> <a href="${place.websiteUri ? place.websiteUri : place.googleMapsUri}"
class="hover:cursor-pointer hover:underline hover:text-cyan-400" target="_blank">${place.displayName.text}</a><br>
  <strong>Open Now:</strong> ${
    place.currentOpeningHours?.openNow ? 'Yes' : 'No'
  }<br>
  ${
    place.currentOpeningHours?.weekdayDescriptions?.[
      new Date().getDay()
    ] || ''
  }<br>
  <div class="mt-4 flex overflow-x-auto space-x-4">
    ${place.photos
      ?.map((photo, photoIndex) => {
        const photoReference = photo.name.split('/').pop();
        return `
`;
      })
      .join('')}
  </div>
</div>
`;

      const popup = new Popup({ offset: 25 }).setHTML(popupContent);

      const marker = new Marker({ element: el })
        .setLngLat([place.location.longitude, place.location.latitude])
        .setPopup(popup)
        .addTo(this.map);

      this.markers.push(marker);

      popup.on('open', () => {
        const button = document.getElementById(`remove-point-${index}`);
        if (button) {
          if (!this.editMode) {
            button.classList.add('hidden');
          } else {
            button.classList.remove('hidden');
            button.addEventListener('click', (event) => {
              event.stopPropagation();
              this.removePoint(index);
            });
          }
        }

        const images = document.querySelectorAll(`#image-${index}`);
        images.forEach((image) => {
          image.addEventListener('click', (event) => {
            const target = event.target as HTMLImageElement;
            const imageSrc = target.src;
            this.openImageResizeModal(imageSrc);
          });
        });
      });
    });
  });
}

```

Slika 3.20. Funkcija *setPlaceMarker* unutar *front-end map.component.ts*

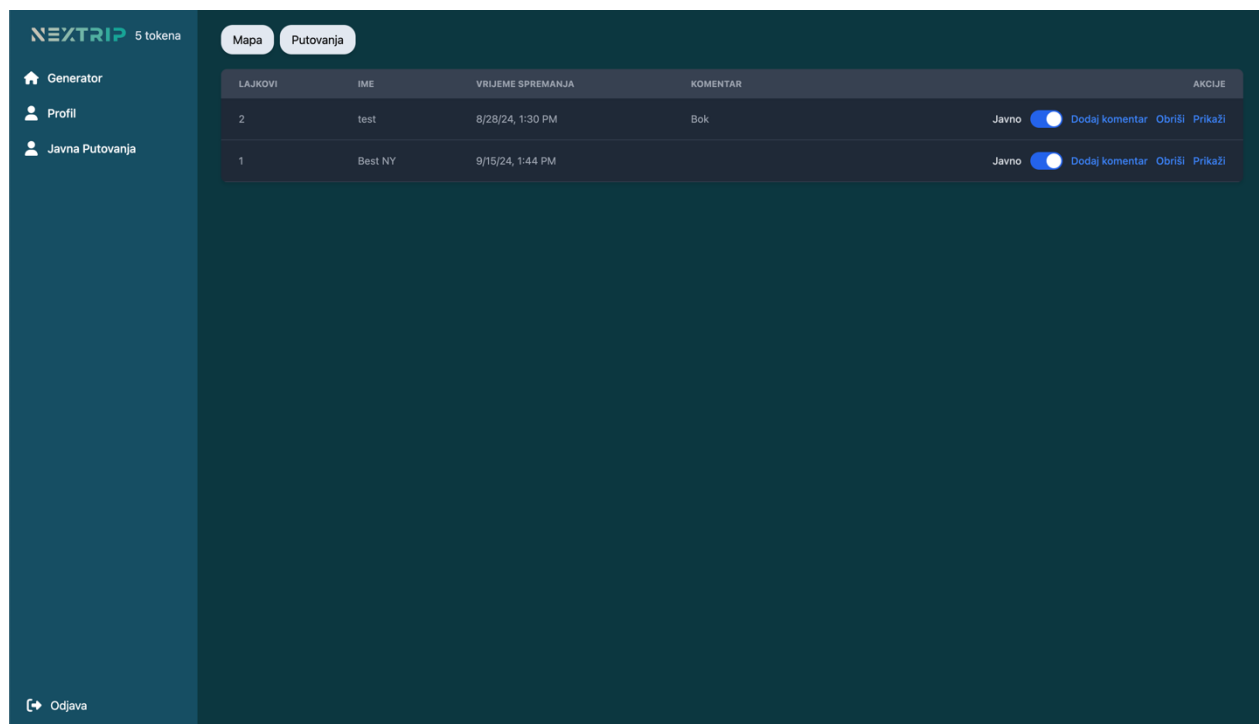
Slika 3.21. prikazuje Informacije o putovanju koji se nalazi na desnoj strani od mape. Funkcija Informacije o putovanju komponente je da prikaze opis svih dobivenih mjesta. Informacije koje sadrži su Ime, ulica, web stranica, je li je otvoreno trenutno, te radne sate.



Slika 3.21. Prikaz Informacije o putovanju

3.5.3. Spremljena putovanja

Slika 3.22. prikazuje *child* komponentu Putovanja koja sadrži sva putovanja koje je korisnik spremio nakon generiranja. Putovanja sadrže koliko *like*-ova ima, ime, vrijeme spremanja, komentar te akcije: Javno koja omogućava da bude vidljiva svim korisnicima aplikacije, Dodaj komentar, Obriši i Prikaži koja prikazuje putovanje na mapi.

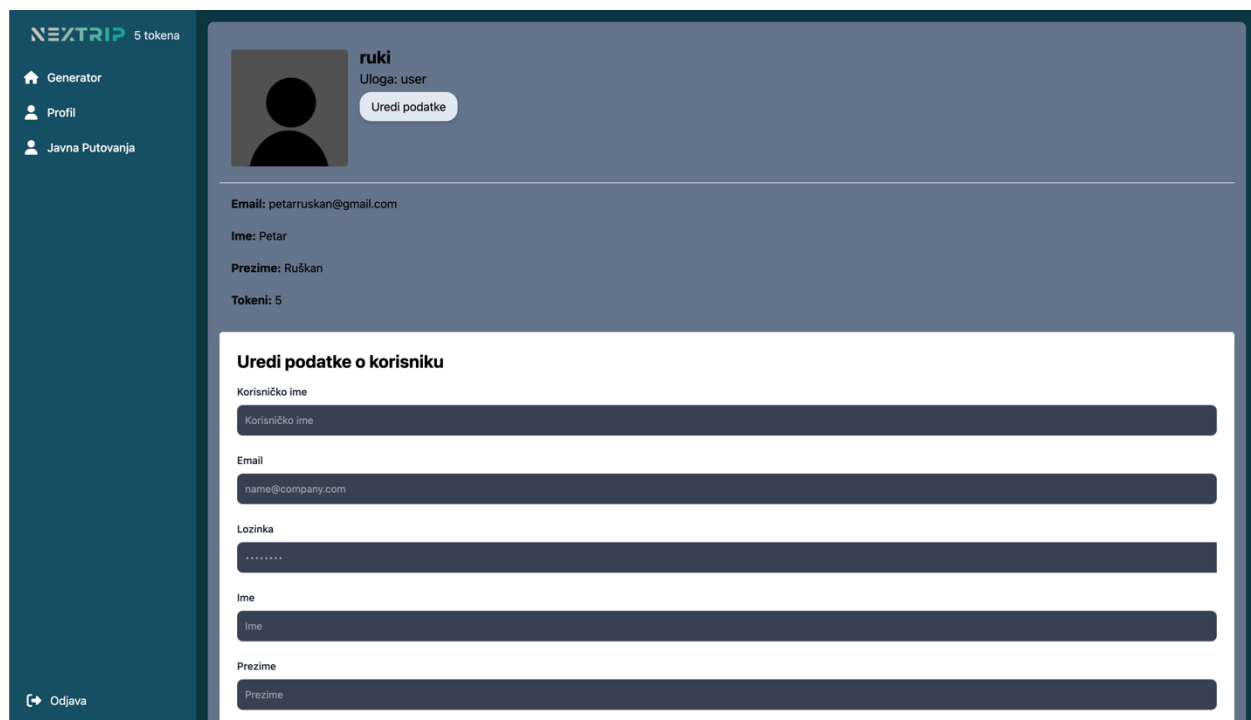


Slika 3.22. Prikaz *child* komponente Putovanja

3.6. Profil

Komponenta Profil unutar aplikacije služi za prikaz i upravljanje korisničkim podacima. Kada korisnik otvori profil, prikazuju mu se osnovni podaci kao što su korisničko ime, uloga, *e-mail* adresa, ime i prezime te broj preostalih tokena. Ove informacije dohvaćaju se s back-end-a i prikazuju na jednostavnom sučelju koje omogućuje korisniku da pregledava svoje podatke na pregledan način. Komponenta je dizajnirana tako da omogućuje korisniku jednostavan pristup vlastitim informacijama te pruža pregled svih važnih detalja vezanih uz račun.

Osim prikaza, komponenta Profil omogućuje korisniku da ažurira svoje podatke. Slika 3.23. prikazuje izgled prikaz komponente te obrazac za promjenu podataka. Promjene se šalju na server gdje se ažuriraju u bazi podataka. Prilikom unosa novih podataka, implementirane su validacije kako bi se osiguralo da korisnik unese ispravne vrijednosti, kao što su validan email ili dovoljno jaka lozinka. Nakon što korisnik unese promjene, dobiva povratnu informaciju o uspješnom ažuriranju podataka. Ova funkcionalnost pruža korisniku kontrolu nad vlastitim profilom i olakšava održavanje ažurnih podataka unutar sustava.



Slika 3.23. Profil komponenta

Slika 3.24. prikazuje implementaciju funkcije koja ažurira korisničke informacije na temelju unesenih podataka. Metoda *updateUserInfo* prima korisnički ID i objekt *updateUserInfoDto*, koji sadrži podatke za ažuriranje. Prvo, poziva se metoda *filterEmptyFields*, koja uklanja sva prazna ili *null* polja iz DTO (engl. *Data Transfer Object*) objekta. Zatim se podatci o korisniku spremaju u bazu.

Ako broj podudaranja u ažuriranju iznosi nula, to znači da korisnik nije pronađen te se baca iznimka *NotFoundException*. U suprotnom, vraća se *boolean* vrijednost ovisno o tome je li broj ažuriranih dokumenata veći od nule. U slučaju greške metoda vraća *false*.

```

async updateUserInfo(
  userId: string,
  updateUserInfoDto: UpdateUserInfoDto,
): Promise<boolean> {
  try {
    const filteredUpdateDto = this.filterEmptyFields(updateUserInfoDto);
    Logger.debug(filteredUpdateDto);

    const updateResult = await this.userModel.updateOne(
      { _id: userId },
      { $set: filteredUpdateDto },
      { runValidators: true },
    );

    if (updateResult.matchedCount === 0) {
      throw new NotFoundException('User not found');
    }

    return updateResult.modifiedCount > 0;
  } catch (error) {
    console.error('Error updating user info:', error);
    return false;
  }
}

private filterEmptyFields(
  updateUserInfoDto: UpdateUserInfoDto,
): Partial<UpdateUserInfoDto> {
  const filteredDto: Partial<UpdateUserInfoDto> = {};

  for (const key in updateUserInfoDto) {
    if (updateUserInfoDto[key] !== null && updateUserInfoDto[key] !== '') {
      filteredDto[key] = updateUserInfoDto[key];
    }
  }

  return filteredDto;
}

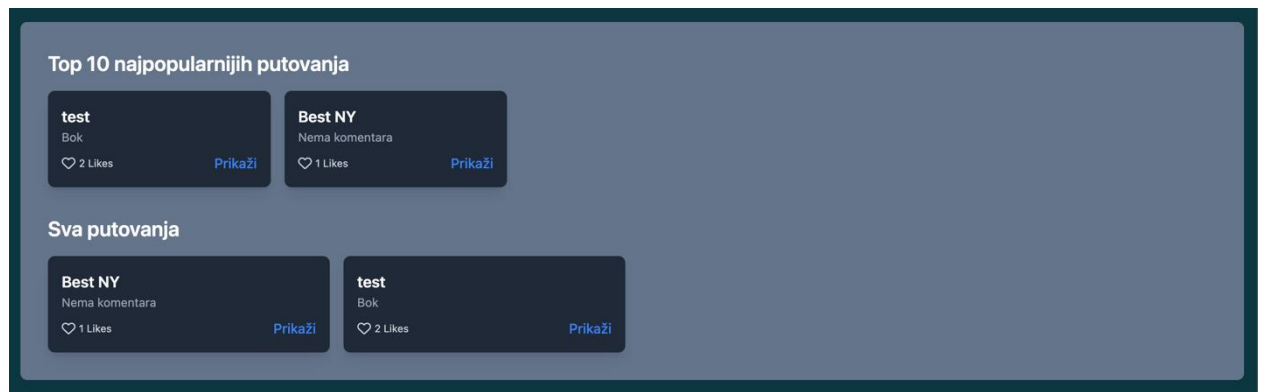
```

Slika 3.24. Funkcije za ažuriranje korisnikovih podataka na *back-end-u*

3.7. Javna mjesta

Slika 3.25. prikazuje komponentu Javna Putovanja koja se sastoji od dva glavna dijela: Top 10 najpopularnijih putovanja i Sva putovanja. Gornji dio prikazuje najpopularnija putovanja koja su dobila najviše lajkova. U ovom primjeru prikazana su dva putovanja s imenima *test* i *Best NY*, uz opis putovanja i broj lajkova. Pored svakog putovanja postoji *link* Prikaži koji omogućava detaljniji prikaz tog putovanja na mapi.

Donji dio prikazuje sva dostupna putovanja koja se prikazuju nasumično svaki put kada se otvori komponenta Javna Putovanja. Poput gornjeg dijela, i ovdje su prikazana dva putovanja s istim nazivima, *Best NY* i *test*, s odgovarajućim brojem lajkova i mogućnošću prikaza putem Prikaži.



Slika 3.25. Komponenta Javna Putovanja

Slika 3.26. prikazuje funkcioniranje *like*-ovanja putovanja. Korisnik može „lajkati“ putovanja ili „unlajkati“ putovanja koje je već „lajkao“. Kôd predstavlja asinkronu funkciju koja ažurira broj „lajkova“ za određeno putovanje. Funkcija prima tri parametra: ID korisnika, ID putovanja i promjena u broju lajkova, koja može biti 1 ili -1. Na početku, funkcija provjerava je li vrijednost promjene ispravna (mora biti 1 ili -1), a ako nije, baca se iznimka.

```

async updateTripLikes(
  userId: string,
  tripId: string,
  change: number,
): Promise<boolean> {
  try {
    if (![1, -1].includes(change)) {
      throw new BadRequestException(
        'Invalid change value. It must be 1 or -1.'
      );
    }

    const userObjectId = new Types.ObjectId(userId);
    const tripObjectId = new Types.ObjectId(tripId);

    const user = await this.userModel.findOne({
      'trips.tripId': tripObjectId,
    }).exec();
    if (!user) {
      throw new NotFoundException(`User with trip ID ${tripId} not found`);
    }

    const tripToUpdate = user.trips.find(trip => trip.tripId.equals(tripObjectId));
    if (!tripToUpdate) {
      throw new NotFoundException(`Trip with ID ${tripId} not found in user's trips`);
    }

    const newLikesCount = tripToUpdate.likes + change;
    const userHasLiked = tripToUpdate.likedBy.includes(userObjectId);

    if (change === 1 && !userHasLiked) {
      tripToUpdate.likes = newLikesCount;
      tripToUpdate.likedBy.push(userObjectId);
    } else if (change === -1 && userHasLiked) {
      tripToUpdate.likes = newLikesCount;
      tripToUpdate.likedBy = tripToUpdate.likedBy.filter(id => !id.equals(userObjectId));
    } else {
      return false;
    }

    await user.save();

    return true;
  } catch (error) {
    if (error instanceof NotFoundException) {
      throw error;
    } else {
      throw new InternalServerErrorException('Error updating trip likes');
    }
  }
}

```

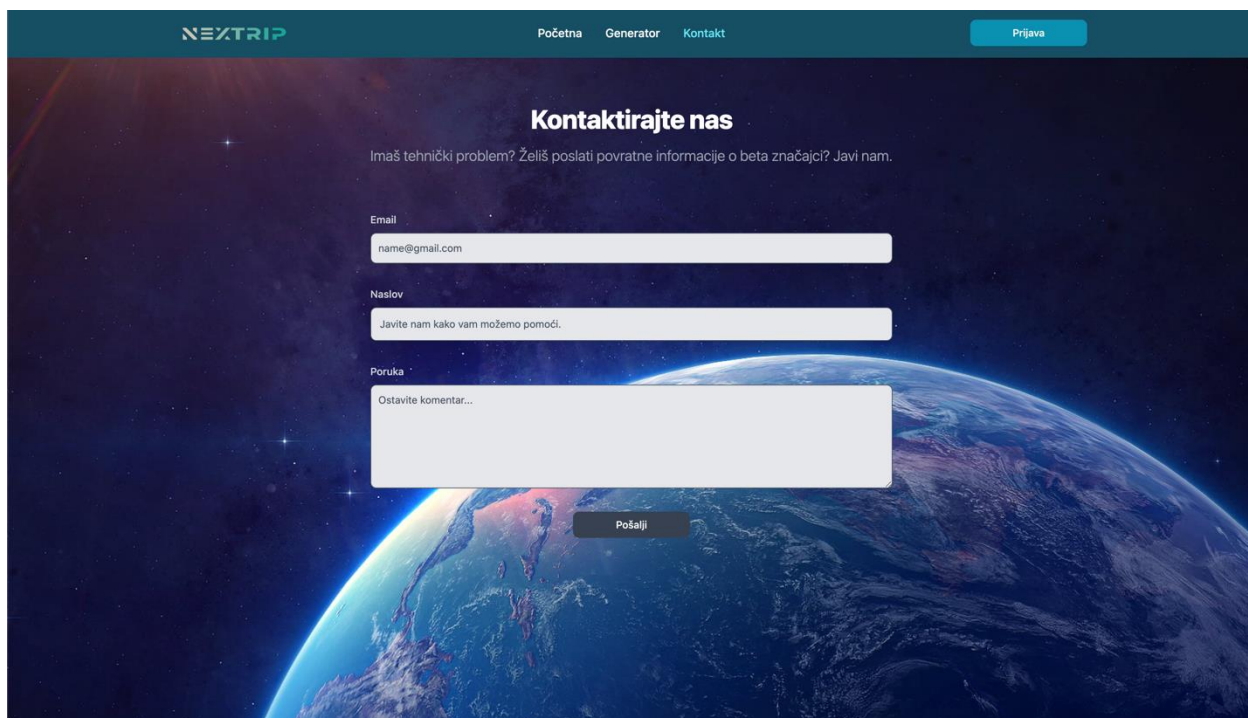
Slika 3.26. Funkcija *updateTripLikes* unutar *back-end generator.service.ts*

Funkcija zatim traži korisnika u bazi podataka koji ima navedeno putovanje u svojoj listi. Ako korisnik nije pronađen, baca se iznimka s porukom da korisnik s tim ID-om putovanja nije pronađen. Zatim se traži konkretno putovanje unutar korisnikovih putovanja, a ako ono ne postoji, baca se iznimka s porukom da to putovanje nije pronađeno.

Ako su korisnik i putovanje uspješno pronađeni, funkcija izračunava novi broj lajkova dodavanjem vrijednosti promjene na trenutni broj lajkova. Ako korisnik nije već „lajkao“ putovanje i pokušava dodati lajk (promjena je 1), ažurira se broj lajkova i ID korisnika se dodaje u listu korisnika koji su lajkali. Ako korisnik želi ukloniti lajk (promjena je -1) i već je lajkao putovanje, broj lajkova se smanjuje i ID korisnika se uklanja iz liste. Nakon što se podaci ažuriraju, funkcija pohranjuje promjene u bazu podataka i vraća *true*. Ako dođe do bilo kakve greške ili neispravnih podataka, baca se odgovarajuća iznimka s porukom o grešci.

3.8. Kontakt stranica

Na slici 3.27. je prikazana Kontakt stranica koja služi za kontaktiranje tehničke podrške ili slanje povratnih informacija o značajkama.



Slika 3.27. Kontakt stranica

Postoje tri glavna polja za unos informacija. Prvo polje je za unos *e-mail* adrese, gdje korisnik treba unijeti svoju *e-mail* adresu. Drugo polje je za unos naslova poruke, a treće polje služi za unos same poruke, gdje korisnik može ostaviti komentar ili detaljnije opisati problem.

Slika 3.28. prikazuje funkciju unutar *back-end* komponente za slanje *mail*-ova. Biblioteka *Nodemailer* je popularna Node.js biblioteka koja omogućuje jednostavno slanje *e-mail*ova iz aplikacija. Ova biblioteka omogućuje slanje e-mailova s unesenim podacima iz obrasca, kao što su korisnička *e-mail* adresa, predmet poruke i sama poruka. Nodemailer podržava različite SMTP (engl. *Simple Mail Transfer Protocol*) servere i pruža fleksibilnost prilikom postavljanja konfiguracija za autentifikaciju, prilagođavanje sadržaja poruka te slanje privitaka, čime se olakšava komunikacija između korisnika i tehničke podrške aplikacije.

A screenshot of a code editor with a dark background and light-colored text. The code is written in TypeScript and defines an asynchronous function named `sendContactForm`. The function takes three parameters: `email` (string), `subject` (string), and `message` (string). It returns a `Promise<string>`. Inside the function, a transporter is created using `nodemailer.createTransport` with the service set to 'gmail' and authentication credentials from the environment. Mail options are defined with the email, subject, and message. A try-catch block attempts to send the email using `transporter.sendMail` and returns a success message or an error message.

```
async sendContactForm(
  email: string,
  subject: string,
  message: string,
): Promise<string> {
  const transporter = nodemailer.createTransport({
    service: 'gmail',
    auth: {
      user: environment.MAILER_USER,
      pass: environment.MAILER_PASS,
    },
  });

  const mailOptions = {
    from: email,
    to: 'nextrip.company@gmail.com',
    subject: subject,
    text: message,
  };

  try {
    await transporter.sendMail(mailOptions);
    return 'Email sent successfully';
  } catch (error) {
    return `Failed to send email: ${error.message}`;
  }
}
```

Slika 3.28. Funkcija *sendContactForm* unutar *back-end* *contact.service.ts*

4. ZAKLJUČAK

Zaključkom ovog završnog rada izrađena je *web* aplikacija Generator Putovanja (Nextrip), koja omogućuje korisnicima jednostavno i intuitivno planiranje putovanja. Aplikacija pruža korisničko sučelje koje omogućava kreiranje personaliziranih itinerera kroz odabir lokacija, uz dodatne opcije poput označavanja putovanja kao privatnog ili javnog te mogućnost dodavanja komentara i interakcije s drugim korisnicima putem lajkova. Implementirane su napredne funkcionalnosti, uključujući autentifikaciju korisnika koristeći JWT, što omogućuje sigurno upravljanje korisničkim računima i podacima.

U pozadini aplikacije koristi se NestJS razvojni okvir za *back-end* razvoj, dok MongoDB služi za pohranu podataka o korisnicima i putovanjima. Struktura baze podataka pažljivo je dizajnirana kako bi omogućila fleksibilnost u dodavanju novih lokacija i jednostavno upravljanje putovanjima. *Front-end* je razvijen koristeći Angular, pružajući korisnicima responzivno i moderno sučelje.

Ova aplikacija omogućuje korisnicima kreiranje itinerera s jasno definiranim koracima i pruža alat za organizaciju putovanja na jednostavan način. Moguća buduća poboljšanja uključuju dodavanje integracija s vanjskim servisima poput API-ja za pretraživanje letova, smještaja ili preporuka za restorane i aktivnosti, što bi dodatno unaprijedilo korisničko iskustvo. Jedno od potencijalnih poboljšanja uključuje implementaciju sustava plaćanja putem kojeg bi korisnici mogli kupovati tokene. Ti tokeni bi se zatim koristili za pristup premium funkcionalnostima, poput kreiranja većeg broja putovanja ili otključavanja dodatnih značajki unutar aplikacije.

Također, korisnička interakcija mogla bi biti poboljšana dodavanjem opcije za chat između korisnika i vlasnika putovanja koje je postavljeno kao javno. Ova funkcionalnost omogućila bi korisnicima da postavljaju dodatna pitanja ili traže savjete od osoba koje su prethodno kreirale putovanje, što bi potaknulo zajedničko planiranje i razmjenu informacija. Takve interakcije bi unaprijedile komunikaciju unutar zajednice korisnika i dodatno obogatile iskustvo korištenja aplikacije, čineći ju dinamičnijom i društveno interaktivnijom platformom.

Razvoj ove aplikacije pokazuje kako moderna tehnologija može značajno olakšati proces planiranja putovanja, stvarajući korisnički orijentirano rješenje koje pojednostavljuje organizaciju i interakciju između putnika.

LITERATURA

- [1] Zhou, X., & Zhang, Z. (2017). A Personalized Travel Planning System Using Data Analytics and Recommendation Algorithms. Proceedings of the 26th International Conference on World Wide Web, 1361-1370. [25.05.2024.]
- [2] Google Trips how to use, dostupno na: <https://nocamerabag.com/blog/review-google-trips-app-for-planning-trips> [26.06.2024]
- [3] TripAdvisor, dostupno na: <https://tripadvisor.mediaroom.com/us-about-us> [27.06.2024]
- [4] Kayak, dostupno na: <https://www.kayak.com/about> [27.06.2024]
- [5] Rx-angular RxJS Overview, dostupno na: <https://www.rx-angular.io> [28.06.2024.]
- [6] Angular.dev Overview, dostupno na: <https://angular.dev/overview> [28.06.2024.]
- [7] Tailwind CSS Overview, dsotupno na: <https://tailwindcss.com/> [28.06.2024]
- [8] NestJS Overview, dostupno na: <https://docs.nestjs.com> [1.07.2024.]
- [9] MongoDB Overview, dostupno na:
https://www.tutorialspoint.com/mongodb/mongodb_overview.html [1.07.2024.]
- [10] Visual Studio Code Overview, dostupno na: <https://code.visualstudio.com/> [17.7.2024]
- [11] MapLibre GL JS Overview, dostupno na: <https://maplibre.org/maplibre-gl-js/docs/>
[29.7.2024]
- [12] Nodemailer documentation Overview, dostupno na: <https://nodemailer.com/> [3.8.2024]

SAŽETAK

Projekt Generator Putovanja (NexTrip) predstavlja web aplikaciju razvijenu za kreiranje i personalizaciju putovanja prema željama korisnika. Aplikacija omogućuje korisnicima da odaberu destinacije, kreiraju itinerere, dodaju komentare i dijele putovanja s drugima. *Back-end* aplikacije razvijen je koristeći NestJS razvojni okvir s bazom podataka MongoDB, dok je *front-end* implementiran pomoću Angulara. Autentifikacija korisnika osigurana je putem JWT tokena, čime se jamči sigurnost korisničkih podataka. Aplikacija omogućuje korisnicima interakciju s javnim putovanjima putem lajkova. Aplikacija nudi intuitivno korisničko sučelje i fleksibilan sustav za planiranje, uz mogućnosti daljnjeg razvoja i proširenja funkcionalnosti.

Ključne riječi: Angular, baza podataka, JWT, MongoDB, NestJs, Nodemailer, web aplikacija

ABSTRACT

Title: Trip generator

The Trip Generator (NexTrip) project is a web application developed for creating and personalizing trips according to users' preferences. The application allows users to select destinations, create itineraries, add comments, and share trips with others. The back-end of the application is developed using the NestJS framework with a MongoDB database, while the front-end is implemented with Angular. User authentication is secured through JWT tokens, ensuring the safety of user data. The application enables users to interact with public trips through likes. It offers an intuitive user interface and a flexible trip planning system, with opportunities for further development and feature expansion.

Keywords: Angular, database, JWT, MongoDB, NestJS, Nodemailer, web application

ŽIVOTOPIS

Petar Ruškan je student završne godine računarstva s strašću za full-stack razvoj. Tijekom svog školovanja na Fakultetu elektrotehnike, računarstva i informacijskih tehnologija, stekao je temeljno do srednje napredno znanje u programskim jezicima C, Java, Python i SQL te samostalno znanje Node.js-a, Angular-a, istražujući ih kroz samostalno učenje i sudjelovanje u projektima. Njegovo akademsko putovanje obuhvaća duboko razumijevanje algoritama i struktura podataka, što je primijenio u praktičnim projektima poput web aplikacija i optimizacije performansi.

Petar Ruškan

Potpis autora

PRILOZI

GITHUB: <https://github.com/CroniX-Business/NEXTRIP>